

Lógica y Representación 1

Funciones - Introducción

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen



PROBLEMA



ALGORITMO



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

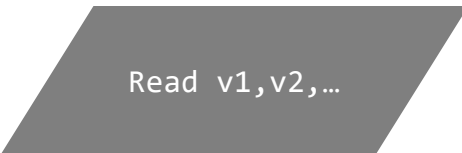
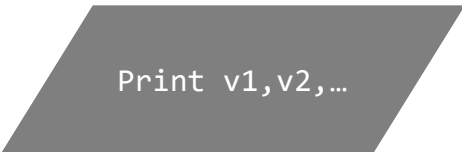


Outputs



Tabla resumen

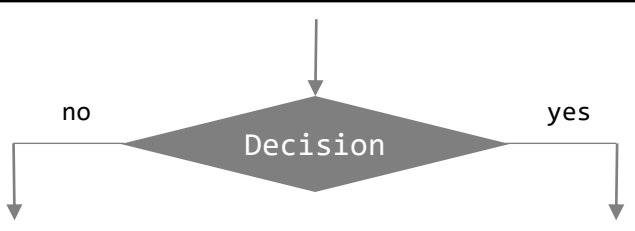
La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Tabla resumen

La siguiente tabla, agrega la nueva el nuevo bloque que permite implementar toma de decisiones en los programas.

Diagrama de bloques	Pseudocodigo	Python
	Si (condicion) Entonces instrucciones verdad Sino instrucciones falsa Fin_Si	<pre>if (condicion): instrucciones verdad else: instrucciones falso</pre>

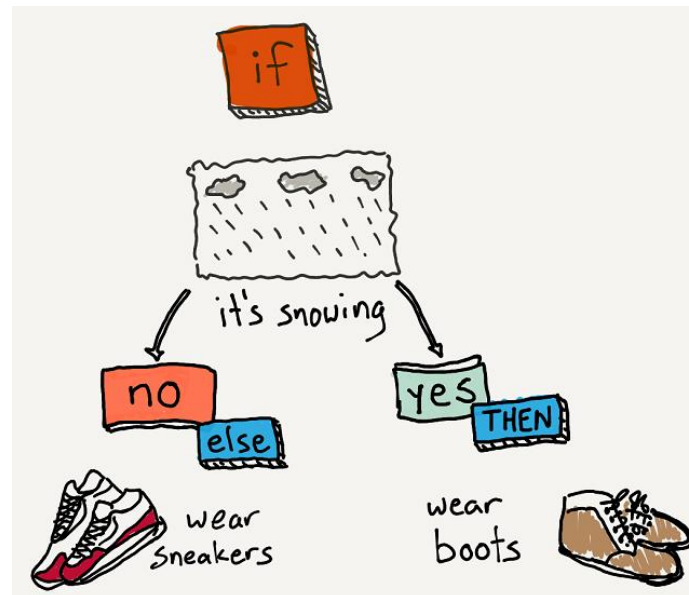


Imagen tomada de CS Intro ([link](#))

Casos en los problemas de decisión lógica

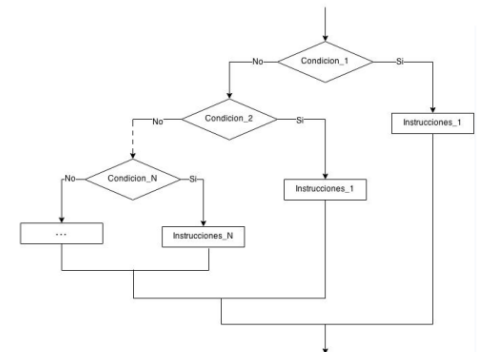
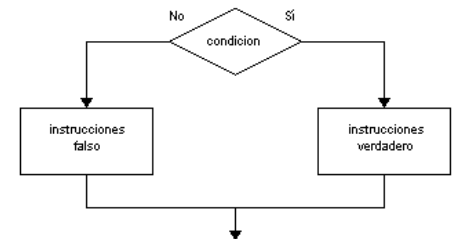
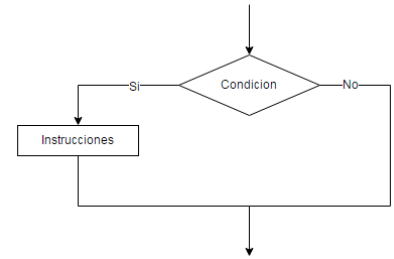


**Alternativas en los
problemas de decisión
lógica**

Alternativa simple

Alternativa doble

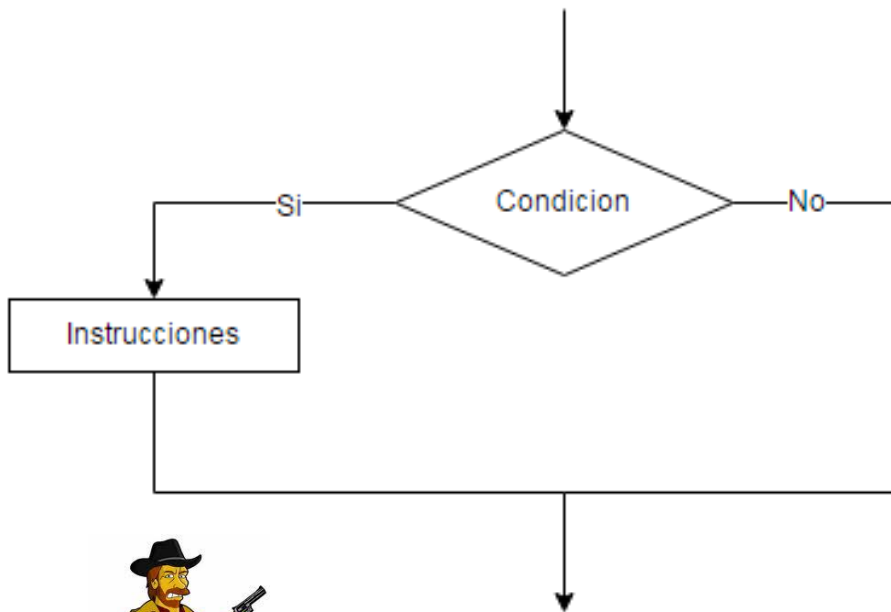
Alternativa múltiple



Alternativa simple

En los problemas de este tipo la condición **falsa** no posee instrucciones.

Diagrama de flujo



Pseudocódigo

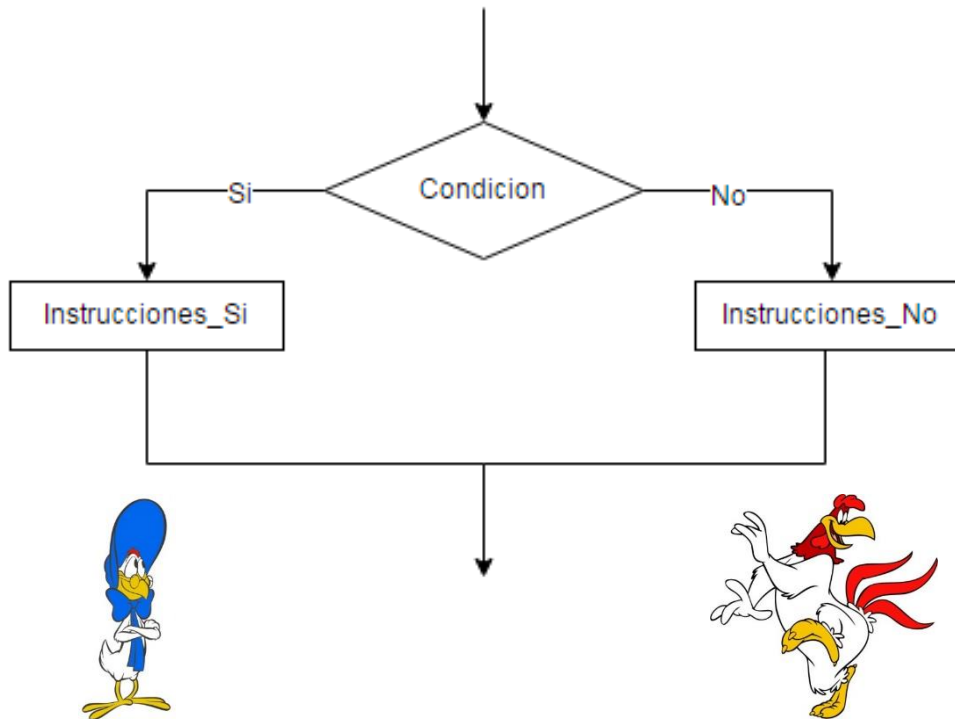
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera
```

Alternativa doble

En los problemas de este tipo, ambos casos (**Falso** y **Verdadero**) poseen instrucciones asociadas.



Pseudocódigo

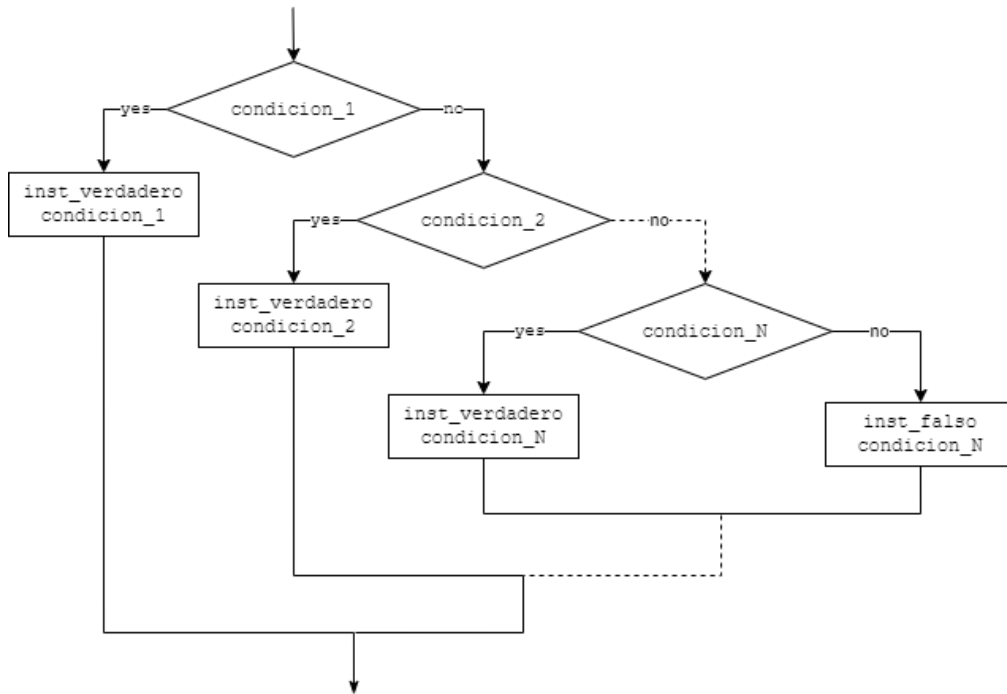
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Sino  
    instrucciones condición falsa  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera  
else:  
    instrucciones condición falsa
```


Alternativa múltiple – Mismo criterio

Diagrama de flujo



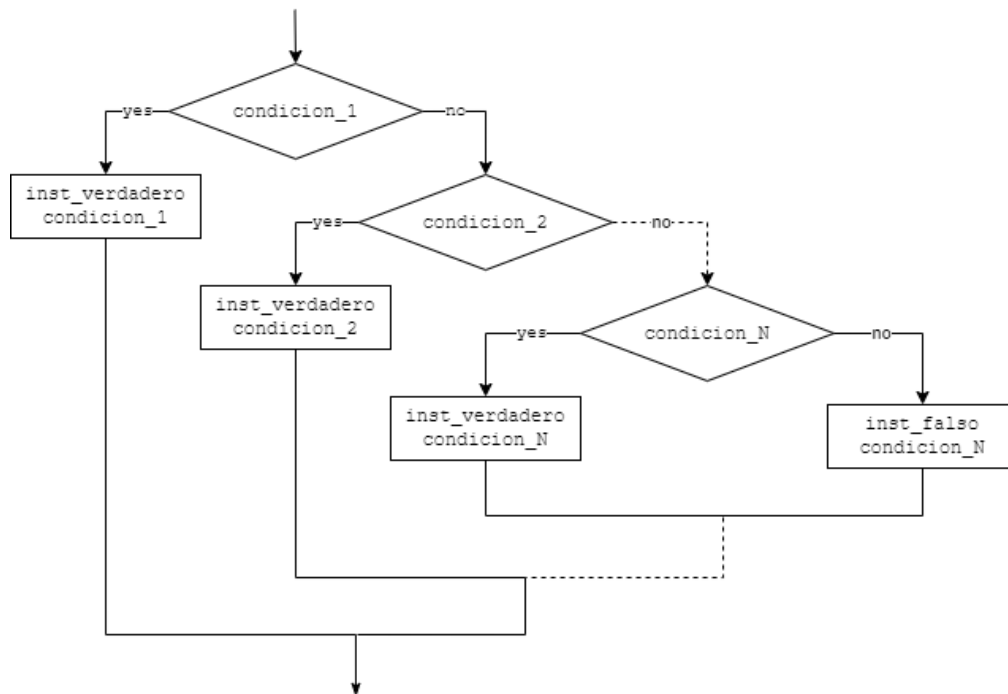
Pseudocódigo

```
Si (condicion_1) Entonces  
    condicion_1 verdad  
Sino  
    Si (condicion_2) Entonces  
        condicion_2 verdad  
    Sino  
        Si (condicion_3) Entonces  
            condicion_3 verdad  
        Sino  
            ...  
            Si (condicion_n) Entonces  
                condicion_n verdad  
            Sino  
                condicion_n falso  
            Fin_Si  
            ...  
        Fin_Si  
    Fin_Si  
Fin_Si
```

Alternativa múltiple – Mismo criterio

python

Diagrama de flujo

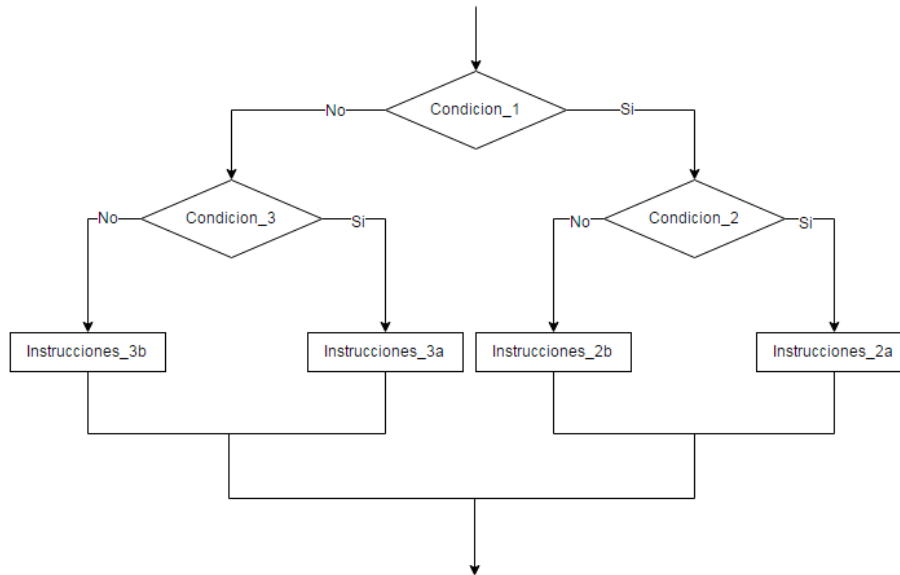


```
if (condicion_1):  
    condicion_1 verdad  
else:  
    if (condicion_2):  
        condicion_2 verdad  
    else:  
        if (condicion_3):  
            condicion_3 verdad  
        else:  
            ...  
            if (condicion_n):  
                condicion_n verdad  
            else:  
                condicion_n falso
```

```
if (condicion_1):  
    condicion_1 verdad  
elif (condicion_2):  
    condicion_2 verdad  
elif (condicion_3):  
    condicion_3 verdad  
...  
elif (condicion_n):  
    condicion_n verdad  
else:  
    condicion_n falso
```

Alternativa múltiple – Diferente criterio

Diagrama de flujo

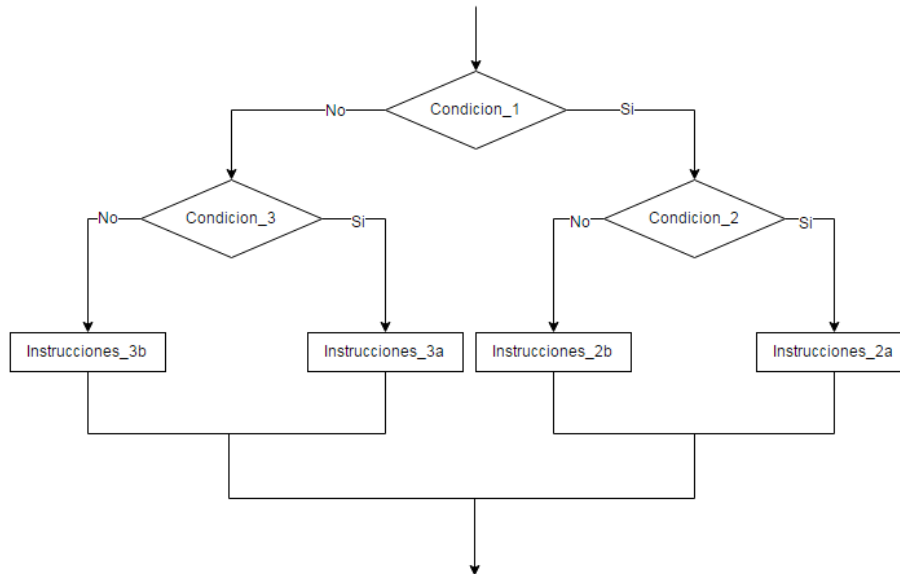


Pseudocódigo

```
Si (condicion_1) Entonces
    Si (condicion_3) Entonces
        condicion_3a verdad
    Sino
        condicion_3b
    Fin_Si
Sino
    Si (condicion_2) Entonces
        condicion_2a
    Sino
        condicion_2b
    Fin_Si
Fin_Si
```

Alternativa múltiple – Diferente criterio

Diagrama de flujo

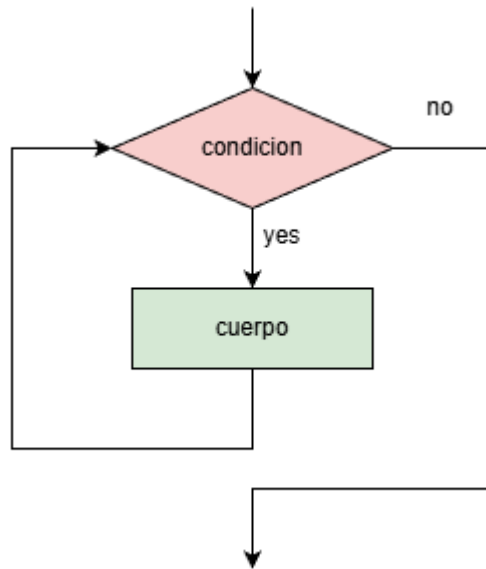


Python

```
if (condicion_1):  
    if (condicion_3):  
        condicion_3a verdad  
    else:  
        condicion_3b  
else:  
    if (condicion_2):  
        condicion_2a  
    else:  
        condicion_2b
```

Ciclo mientras

Diagrama de flujo



Pseudocódigo

```
Mientras (condicion) Haga  
    instrucciones cuerpo  
Fin_Mientras
```

Python

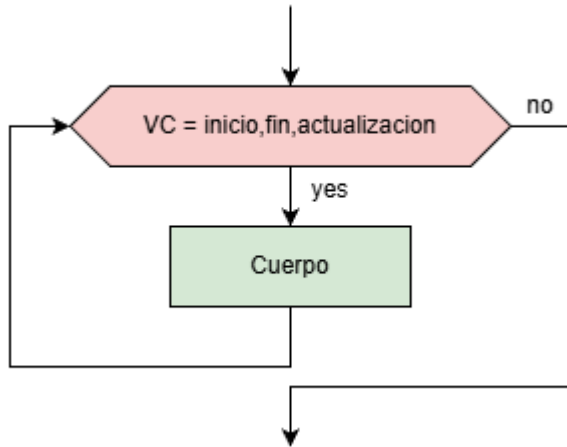
```
while (condicion):  
    instrucciones cuerpo
```

Observaciones importantes:

- Note que aunque la inicialización es importante para el ciclo, es previa a este (por eso no aparece).
- Dentro del cuerpo del ciclo están las instrucciones propias de lo que en este se debe realizar y la actualización de la variable de condición

Ciclo Para

Diagrama de flujo



Pseudocódigo

```
Para (vc = inicio, fin, actualizacion) Haga  
    instrucciones cuerpo  
Fin_Para
```

Python

```
for vc in range(inicio, fin + 1, actualizacion):  
    instrucciones cuerpo
```

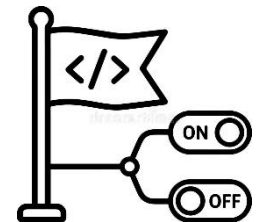
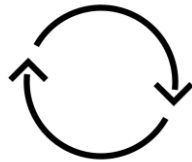
Observaciones importantes:

- A diferencia del ciclo **Mientras**, el ciclo **Para** se caracteriza por que la inicialización, la condición de control (o parada) y la actualización de la variable de control (**VC**) se definen en la cabecera del ciclo.
- Lo anterior hace que la actualización sea un proceso automático, lo cual lo hace la opción mas apropiada para ciclos donde el numero de iteraciones es conocido.
- En python la implementación del **Para**, es diferente a la de lenguajes como C++ y Java, por lo que hay que tener en cuenta unos detalles adicionales que veremos luego.

Tipos de variables útiles en los ciclos

Todo ciclo suele apoyarse en algunos tipos de variables que cumplen roles específicos:

- **Variable de control:** se evalúa en la condición del ciclo para decidir si continúa o termina
- **Contador:** lleva la cuenta de repeticiones; se actualiza en una cantidad constante.
- **Acumulador:** acumula el resultado de una operación que se repite; se actualiza en un valor variable.
- **Bandera:** variable de dos valores excluyentes usada para controlar el ciclo o recordar un estado dentro de él.
- **Centinela:** valor especial, fuera del rango válido de los datos, que señala el fin de la entrada.



Recapitulando lo visto

Tipos de ciclos

Iteración desconocidas

Preguntar antes de cada iteración (uso de bandera)

```
...  
Leer(hay_est)  
Mientras (hay_est == 's') Haga  
  Leer(nota)  
  ...  
  Leer(hay_est)  
Fin_Mientras  
...
```

Usando registro centinela

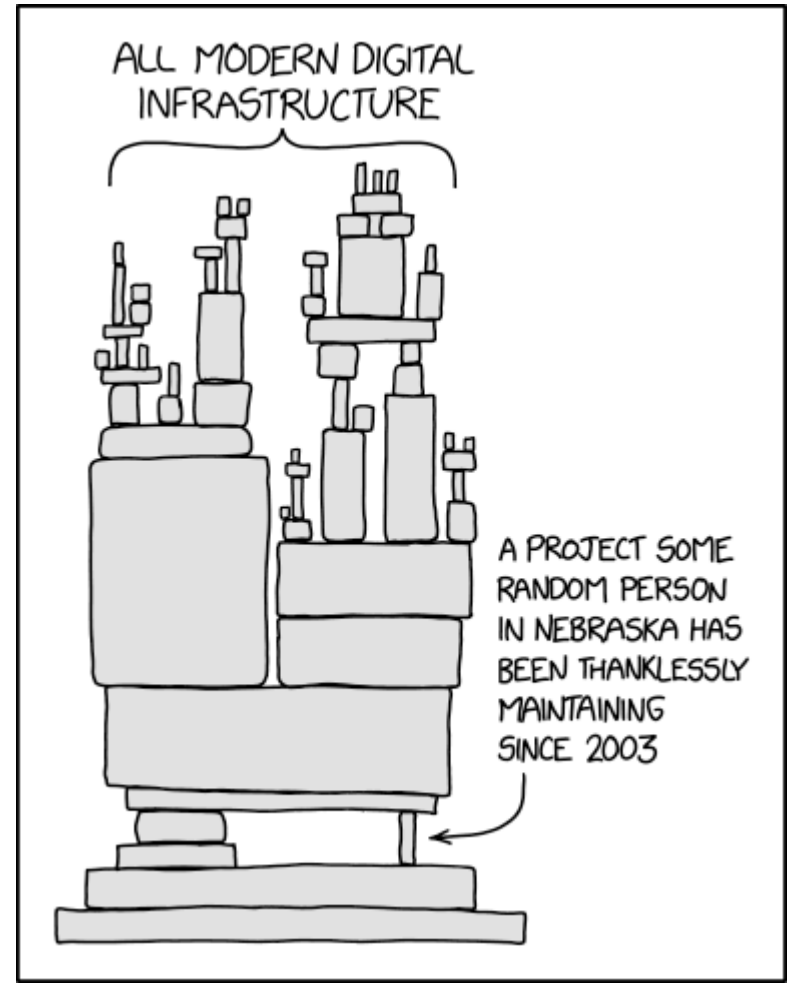
```
...  
Leer(nota)  
Mientras (nota != -1) Haga  
  ...  
  Leer(nota)  
Fin_Mientras  
...
```

Iteraciones conocidas

```
i = 0  
...  
Mientras (i < N) Haga  
  Leer(nota)  
  ...  
  i = i + 1  
Fin_Mientras  
...
```


Introducción a las funciones

```
int getRandomNumber()  
{  
    return 4; // chosen by fair dice roll.  
              // guaranteed to be random.  
}
```



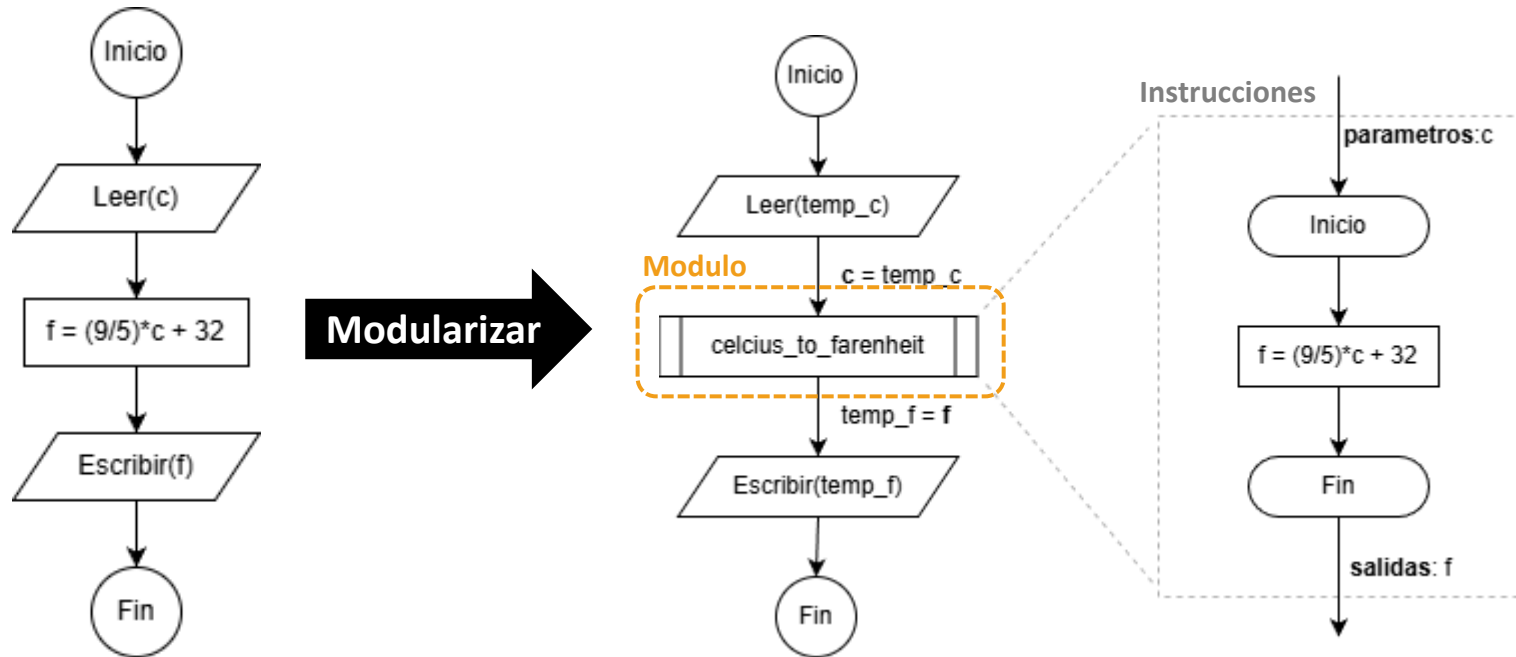
Módulos, subrutinas, procedimientos, funciones o métodos

Modulo:

- Porción de código que realiza una tarea específica.
- También se conoce como: subrutina, procedimiento, función o método.

Modularizar:

- Dividir un programa grande en módulos.

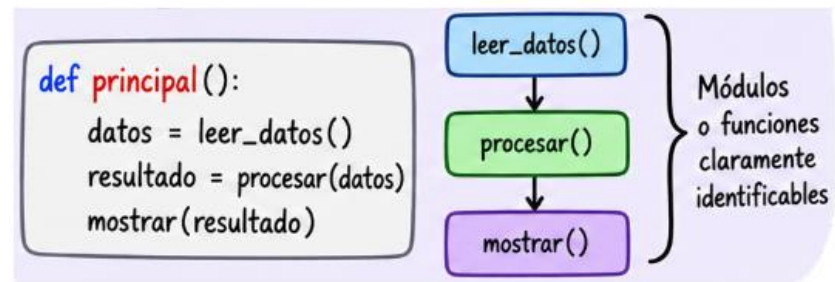
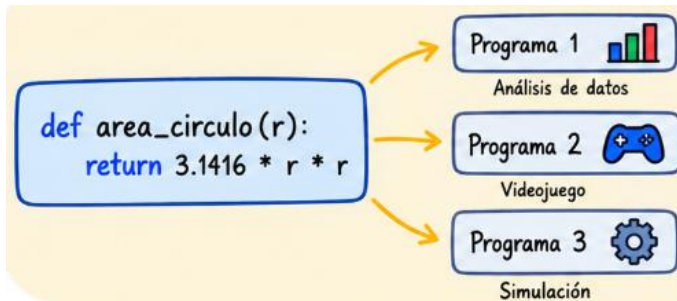
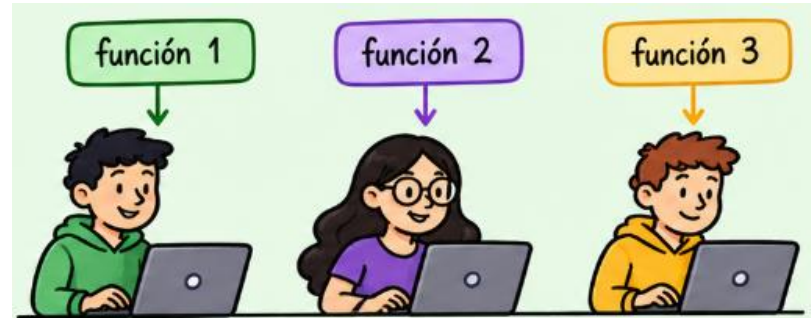
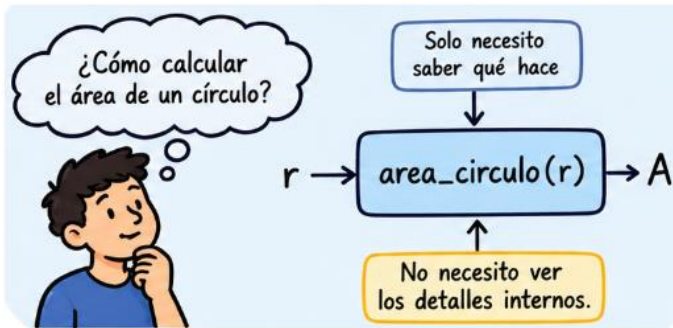


```
c = float(input("Ingrese los °C: "))  
f = (9.0/5)*c + 32  
print(c,"°C =",f, "°F")
```

[Link simulación](#)

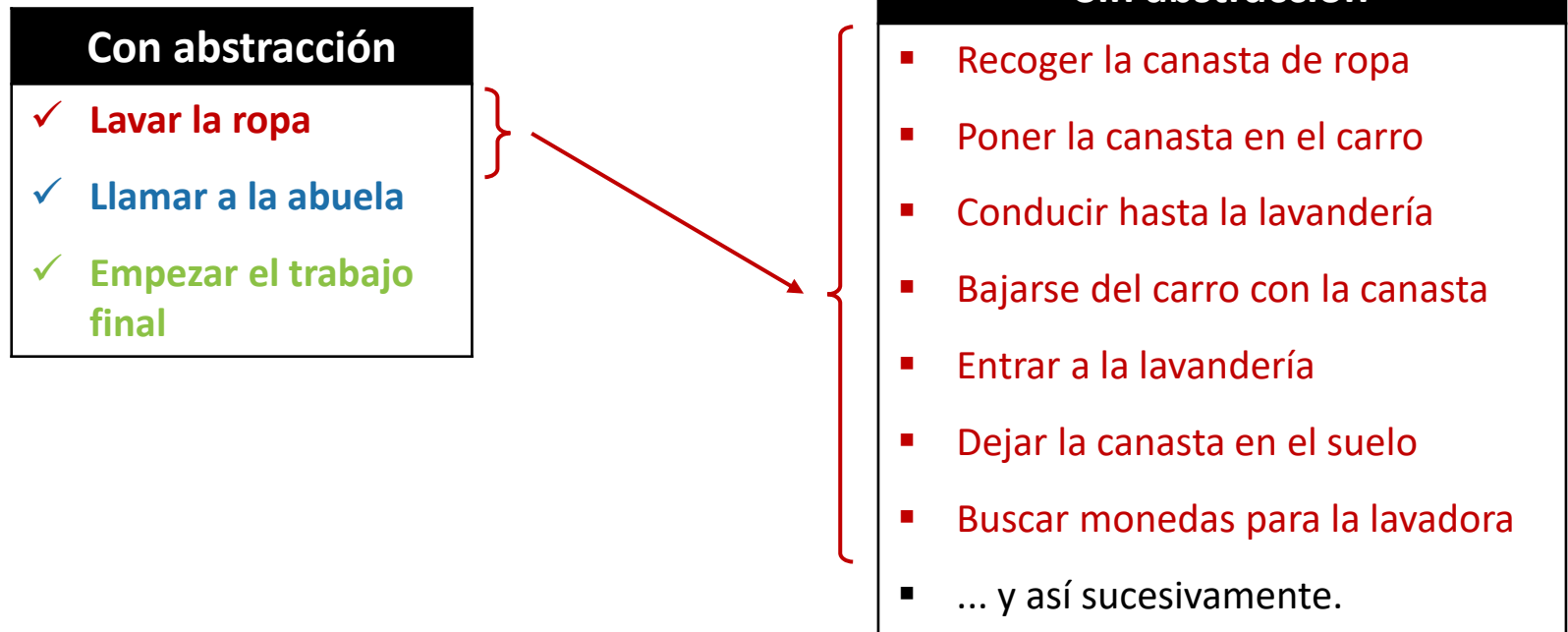
Ventajas de la modularización

- Proporciona abstracción.
- Permite el trabajo simultáneo de múltiples programadores.
- Permite el reuso de código.
- Facilita la identificación de estructuras mas fácilmente.



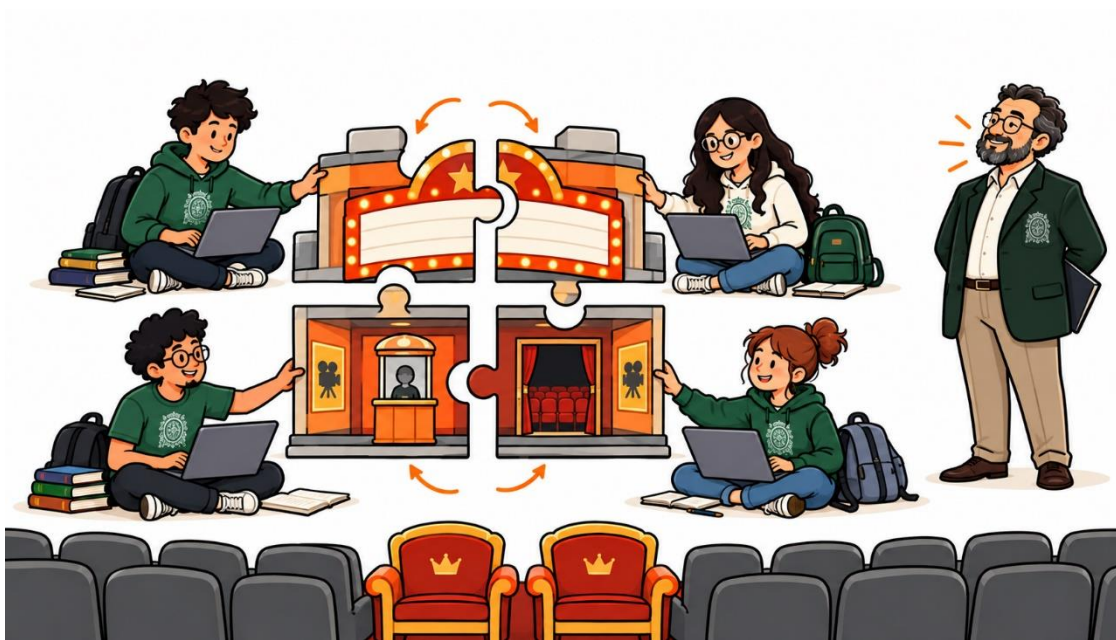
Abstracción

- Permite enfocarse en las propiedades importantes ignorando los detalles no esenciales.
- Evita detalles de bajo nivel y usa un enfoque de alto nivel
- Hace que tareas complejas parezcan simples.
- **Ejemplo:** Lista de tareas pendientes (To Do List)



Trabajo simultaneo

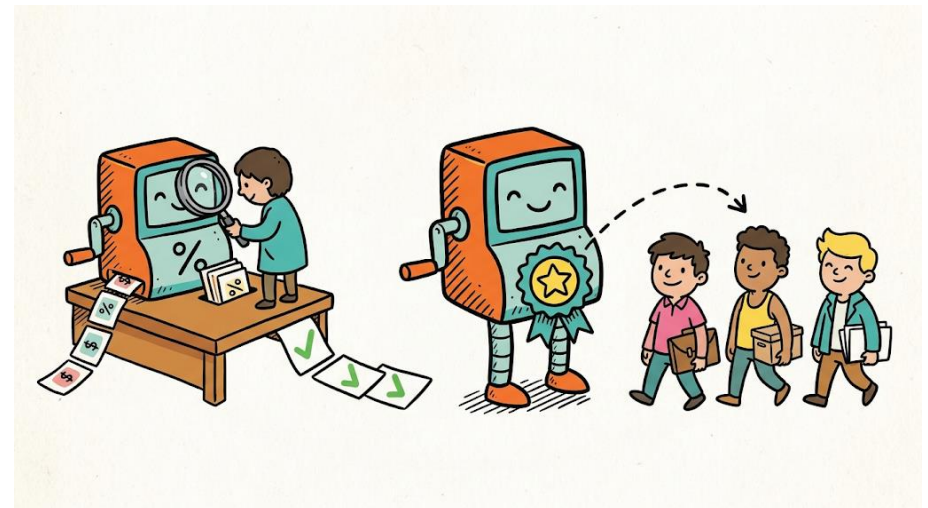
- Los proyectos de programación grandes pueden ser divididos en módulos.
- Los módulos pueden ser escritos por diferentes programadores.
- El tiempo de desarrollo se reduce significativamente.



Reutilización y confiabilidad

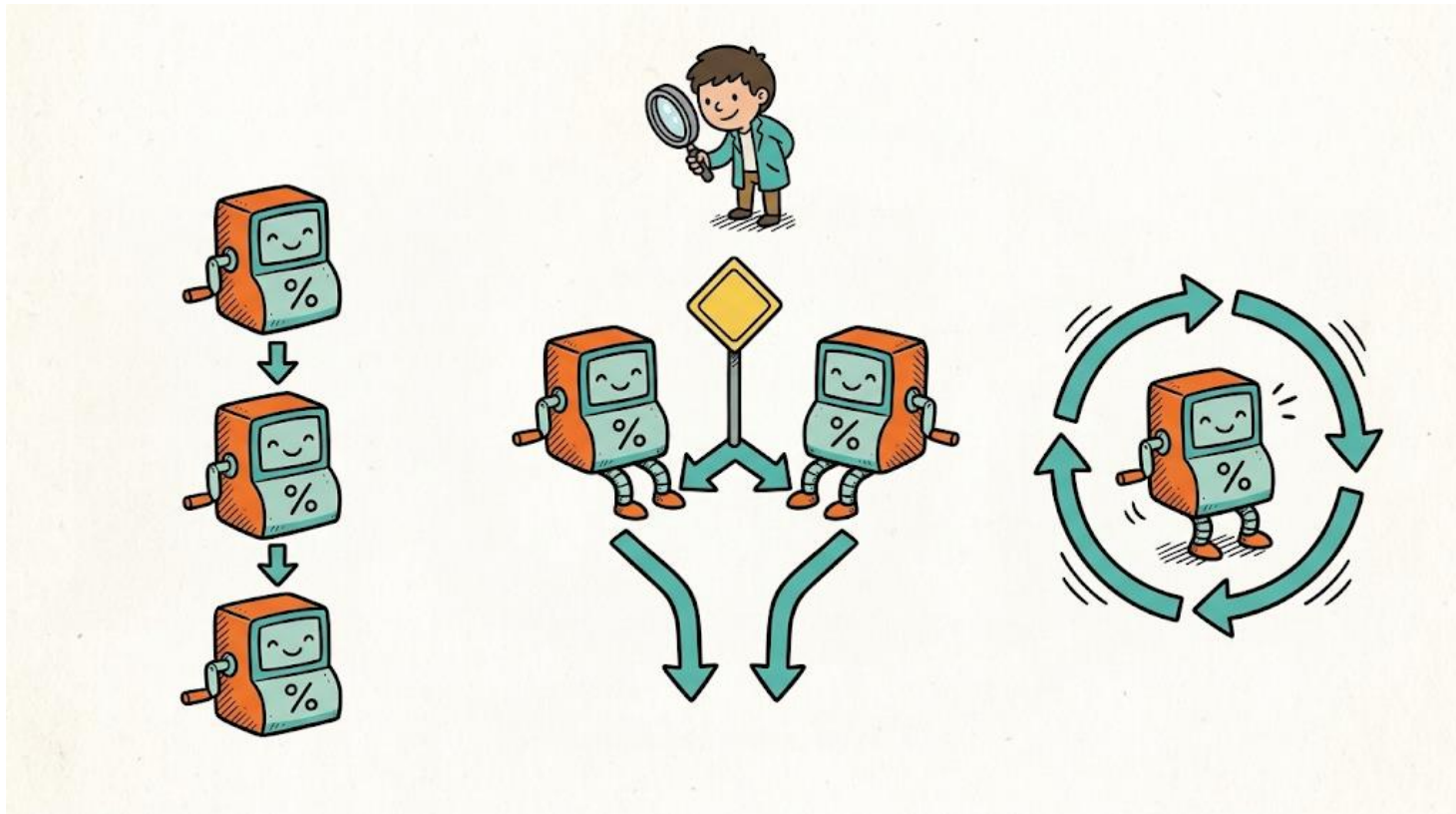
Al modularizar, es posible aprovechar código previamente creado para hacer nuevas cosas:

- **Reutilización:** capacidad de usar módulos en una variedad de aplicaciones.
- **Confiabilidad:** garantía de que un módulo ha sido probado y ha demostrado funcionar correctamente



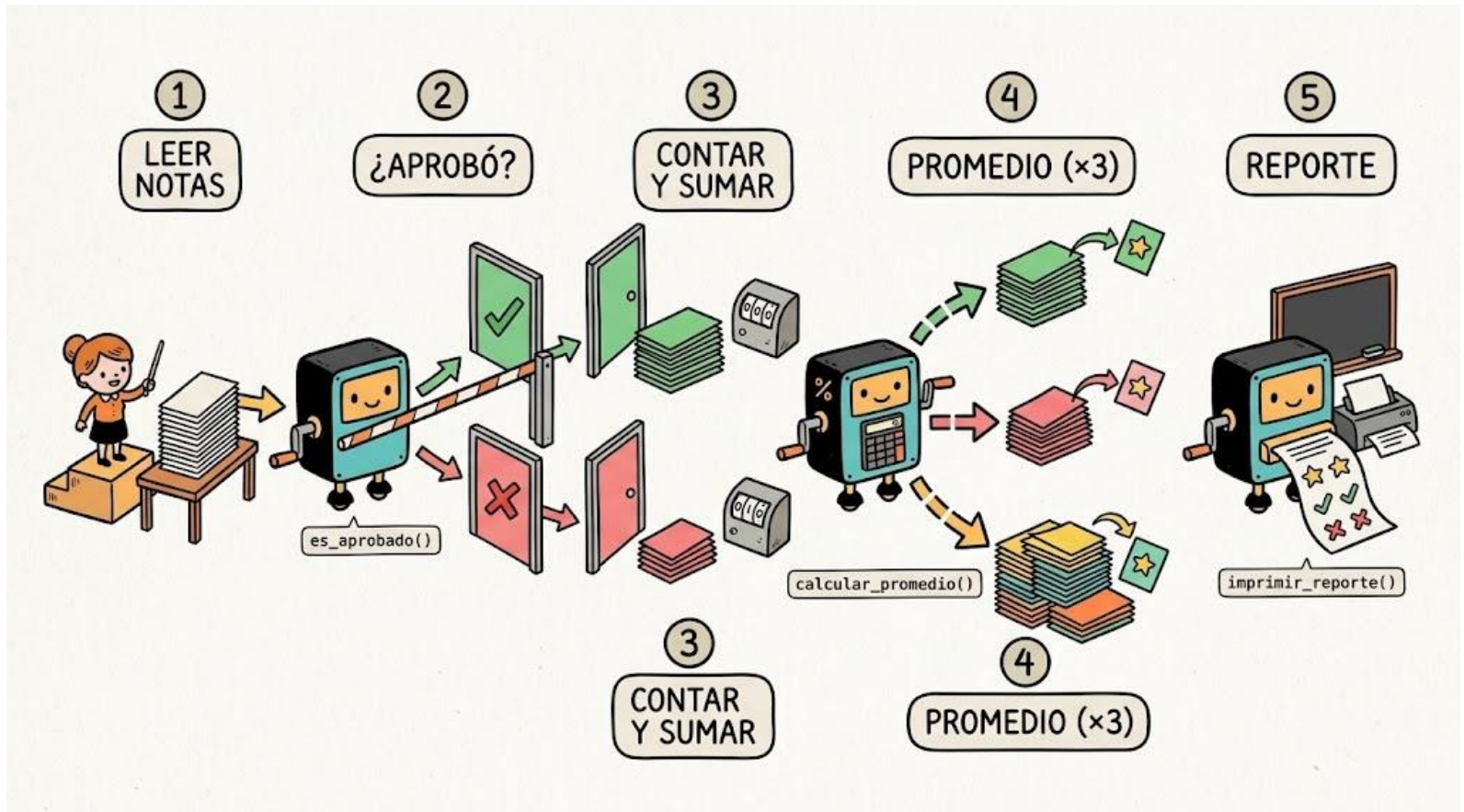
Facilitando la identificación de estructuras

- Agrupar varias tareas en módulos puede facilitar que los programadores principiantes:
 - Determinen si un programa es estructurado.
 - Identifiquen las estructuras de un programa.
- Los programadores experimentados modularizan por la abstracción, la facilidad para dividir el trabajo y la reutilización.



Modularizando un programa

- La mayoría de los programas contienen un **módulo principal**
 - Contiene la **lógica principal**
 - Accede a otros módulos o subrutinas.



Nombrando módulos

- Para nombrar módulos (funciones) se recomienda seguir las siguientes recomendaciones:
 - El nombre debe ser de una sola palabra.
 - El nombre debe ser significativo.
 - Van seguidos de un par de paréntesis.
- **Ejemplo:** Nombres válidos e inválidos para un módulo que calcula el salario bruto de un empleado

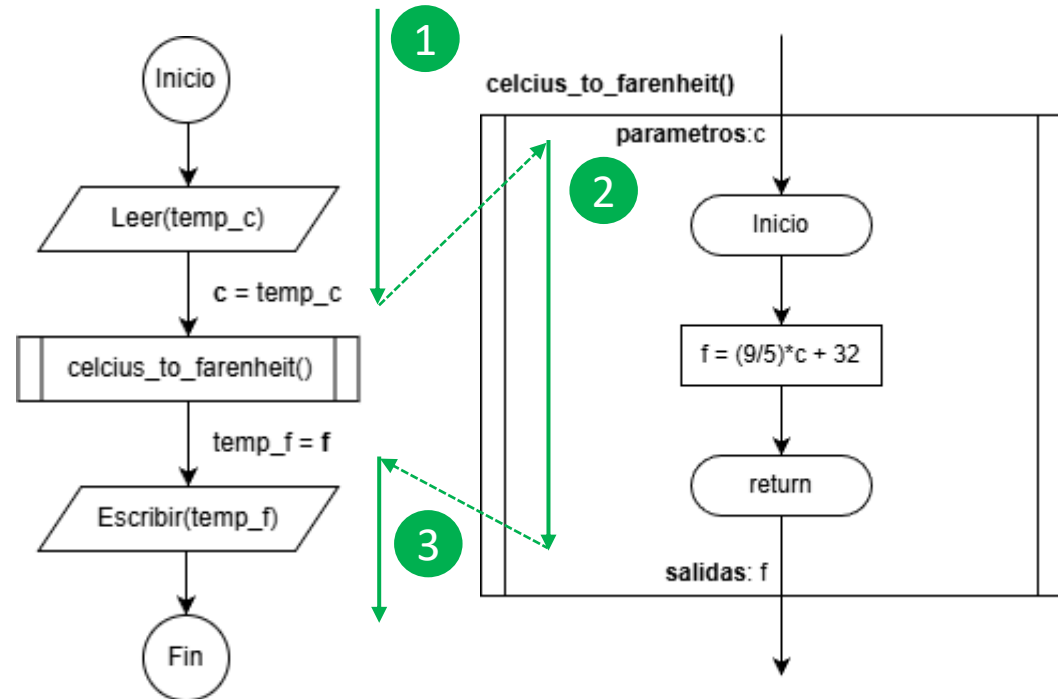
Nombres sugeridos	Comentarios
<code>calcular_salario_bruto()</code>	Bueno
<code>calcular_bruto()</code>	Bueno: la mayoría interpretaría "bruto" como abreviatura de "salario bruto"
<code>cal_sal_br()</code>	Válido , pero críptico
<code>calcular_salario_bruto_para_un_empleado()</code>	Válido , pero torpe
<code>calcular bruto()</code>	No válido: contiene un espacio
<code>calcularsalariobruto()</code>	Válido , pero difícil de leer sin separadores

Función llamadora y función llamada

- **Función llamadora:** la que usa (llama a) otra función.
- **Función llamada:** la que es usada por otra.
- Flujo de ejecución de un programa con funciones:
 - Al llamar a una función, el control pasa a esta (función llamada)
 - Al terminar (**return**), el control vuelve a quien la llamó (función llamadora).

```
def celcius_to_fahrenheit(c):  
    f = (9.0/5)*c + 32  
    return f  
  
temp_c = float(input("Ingresa los °C: "))  
temp_f = celcius_to_fahrenheit(temp_c)  
print(temp_c,"°C =", temp_f,"°F")
```

[Link simulación](#)



Cohesión funcional: una función, una tarea

- Saber cuándo dividir una función en otras es un arte, pero hay una guía
- **Buena práctica:** reunir en una función las instrucciones que contribuyen a **una sola tarea**
- **Cohesión funcional:** grado en que las instrucciones de una función contribuyen a la misma tarea
- **Regla práctica:** si para describir lo que hace una función necesita usar la palabra "y", probablemente son varias funciones

```
# Baja cohesión: lee, calcula y muestra
def promedio_de_notas():
    a = float(input("Nota 1: "))
    b = float(input("Nota 2: "))
    print((a + b) / 2)
```

```
# Programa principal
promedio_de_notas()
```

[Link simulación](#)

```
# Alta cohesión: una tarea por función
def leer_nota(mensaje):
    return float(input(mensaje))
```

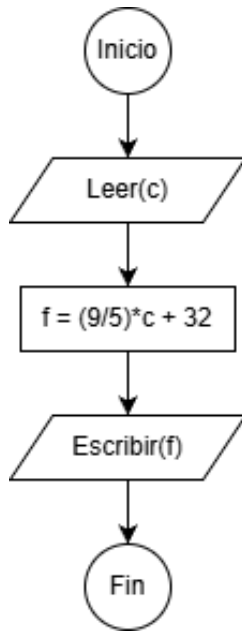
```
def calcular_promedio(a, b):
    return (a + b) / 2
```

```
# Programa principal
nota1 = leer_nota("Nota 1: ")
nota2 = leer_nota("Nota 2: ")
print(calcular_promedio(nota1, nota2))
```

[Link simulación](#)

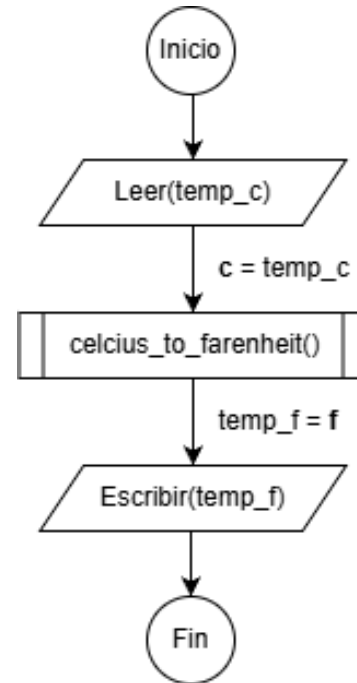
Comparación

- A continuación, se muestra la comparación entre trabajar un programa **sin módulos y con módulos**.



```
temp_c = float(input("Ingrese los °C: "))
temp_f = (9 / 5) * temp_c + 32
print(f"{temp_c} °C = {temp_f} °F")
```

[Link simulación](#)



```
def celsius_a_fahrenheit(c):
    f = (9 / 5) * c + 32
    return f
```

```
temp_c = float(input("Ingrese los °C: "))
temp_f = celsius_a_fahrenheit(temp_c)

print(f"{temp_c} °C = {temp_f} °F")
```

[Link simulación](#)

Análisis – Ejemplo calificaciones

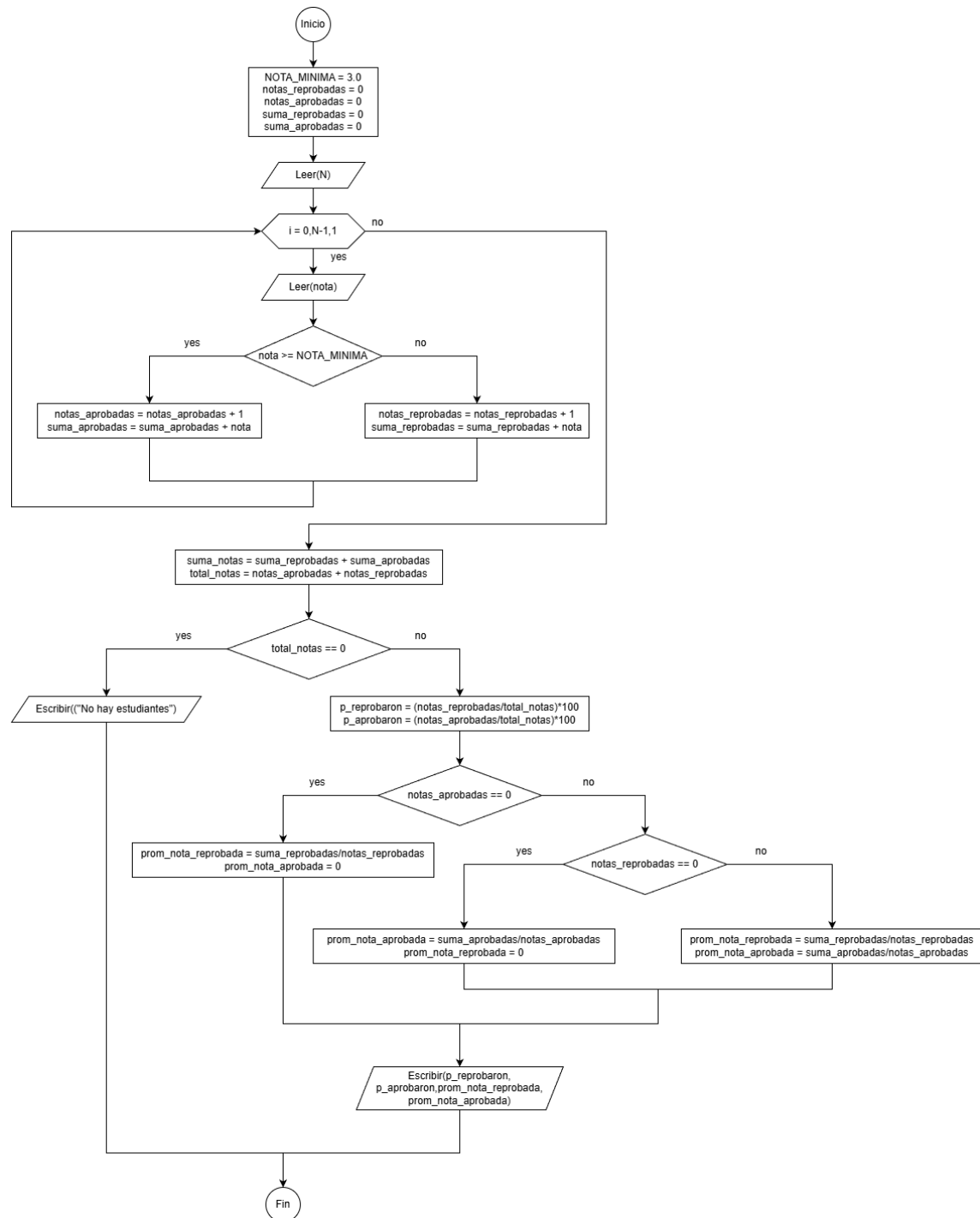
Hacer un programa que lea primero el número de estudiantes de un curso (**N**), y luego lea las **N** notas correspondientes (valores decimales entre 0.0 y 5.0), una a una.

Considere que la nota mínima aprobatoria es 3.0: una nota es aprobada si es mayor o igual a 3.0, y reprobada si es menor a 3.0.

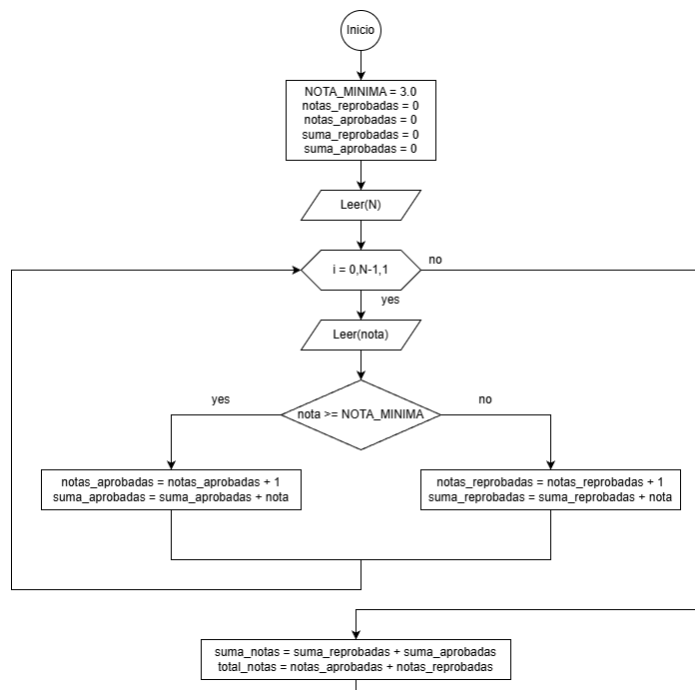
Al finalizar la lectura, el programa deberá entregar:

- El porcentaje de estudiantes aprobados.
- El porcentaje de estudiantes reprobados.
- El promedio de las notas aprobadas.
- El promedio de las notas reprobadas.
- El promedio de las notas del curso.

Ejemplo Calificaciones (No modular)

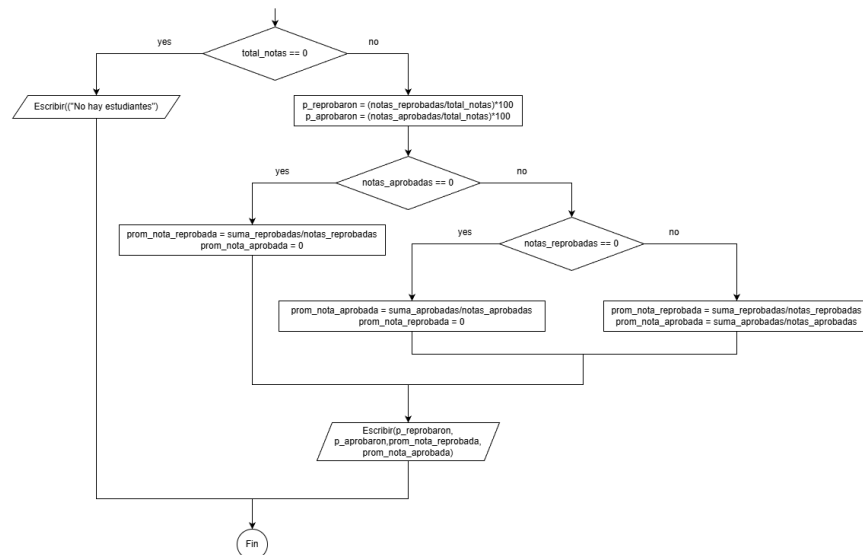


Ejemplo calificaciones



```

NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
...
  
```



```

...
if total_notas == 0:
    print("No hay estudiantes")
else:
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    if (notas_aprobadas == 0):
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = 0
    elif (notas_reprobadas == 0):
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
        prom_nota_reprobada = 0
    else:
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/total_notas
    print(f"Porcentaje aprobados: {p_aprobaron}%")
    print(f"Porcentaje reprobados: {p_reprobaron}%")
    print(f"Promedio reprobados: {prom_nota_reprobada}")
    print(f"Promedio aprobados: {prom_nota_aprobada}")
    print(f"Promedio de notas: {prom}")
  
```

Ejemplo calificaciones

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Prueba de escritorio

Realizar la prueba de escritorio para verificar que el algoritmo este bien planteado

Nota: Este programa así como esta planeado tiene aun un bug grave. Encuéntrelo y corrijalo.

```
NOTA_MINIMA = 3.0
notas_reprobadas = 0
notas_aprobadas = 0
suma_reprobadas = 0
suma_aprobadas = 0
N = int(input('Ingrese el número de estudiantes: '))
for i in range(N):
    nota = float(input('Ingrese la nota: '))
    if nota >= NOTA_MINIMA:
        notas_aprobadas = notas_aprobadas + 1
        suma_aprobadas = suma_aprobadas + nota
    else:
        notas_reprobadas = notas_reprobadas + 1
        suma_reprobadas = suma_reprobadas + nota
suma_notas = suma_reprobadas + suma_aprobadas
total_notas = notas_aprobadas + notas_reprobadas
if total_notas == 0:
    print("No hay estudiantes")
else:
    p_aprobaron = (notas_aprobadas/total_notas)*100
    p_reprobaron = (notas_reprobadas/total_notas)*100
    if (notas_aprobadas == 0):
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = 0
    elif (notas_reprobadas == 0):
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
        prom_nota_reprobada = 0
    else:
        prom_nota_reprobada = suma_reprobadas/notas_reprobadas
        prom_nota_aprobada = suma_aprobadas/notas_aprobadas
    prom = suma_notas/total_notas
    print(f"Porcentaje aprobados: {p_aprobaron}%")
    print(f"Porcentaje reprobados: {p_reprobaron}%")
    print(f"Promedio reprobados: {prom_nota_reprobada}")
    print(f"Promedio aprobados: {prom_nota_aprobada}")
    print(f"Promedio de notas: {prom}")
```


Ejemplo calificaciones

Resultados esperados

Numero de estudiantes	5
Notas	4.0, 2.5, 3.0, 1.5, 4.8
Aprobaron	(3) 60%
Reprobaron	(2) 40%
Promedio nota aprobada	3.93
Promedio nota reprobada	2.0
Promedio general	3.16

Resultado de ejecución

```
Ingrese el número de estudiantes: 5
Ingrese la nota del estudiante 1: 4
Ingrese la nota del estudiante 2: 2.5
Ingrese la nota del estudiante 3: 3.0
Ingrese la nota del estudiante 4: 1.5
Ingrese la nota del estudiante 5: 4.8
Porcentaje de estudiantes que aprobaron: 60.0%
Porcentaje de estudiantes que reprobaron: 40.0%
Promedio de notas reprobadas: 2.0
Promedio de notas aprobadas: 3.9333333333333336
Promedio general de notas: 3.16
```

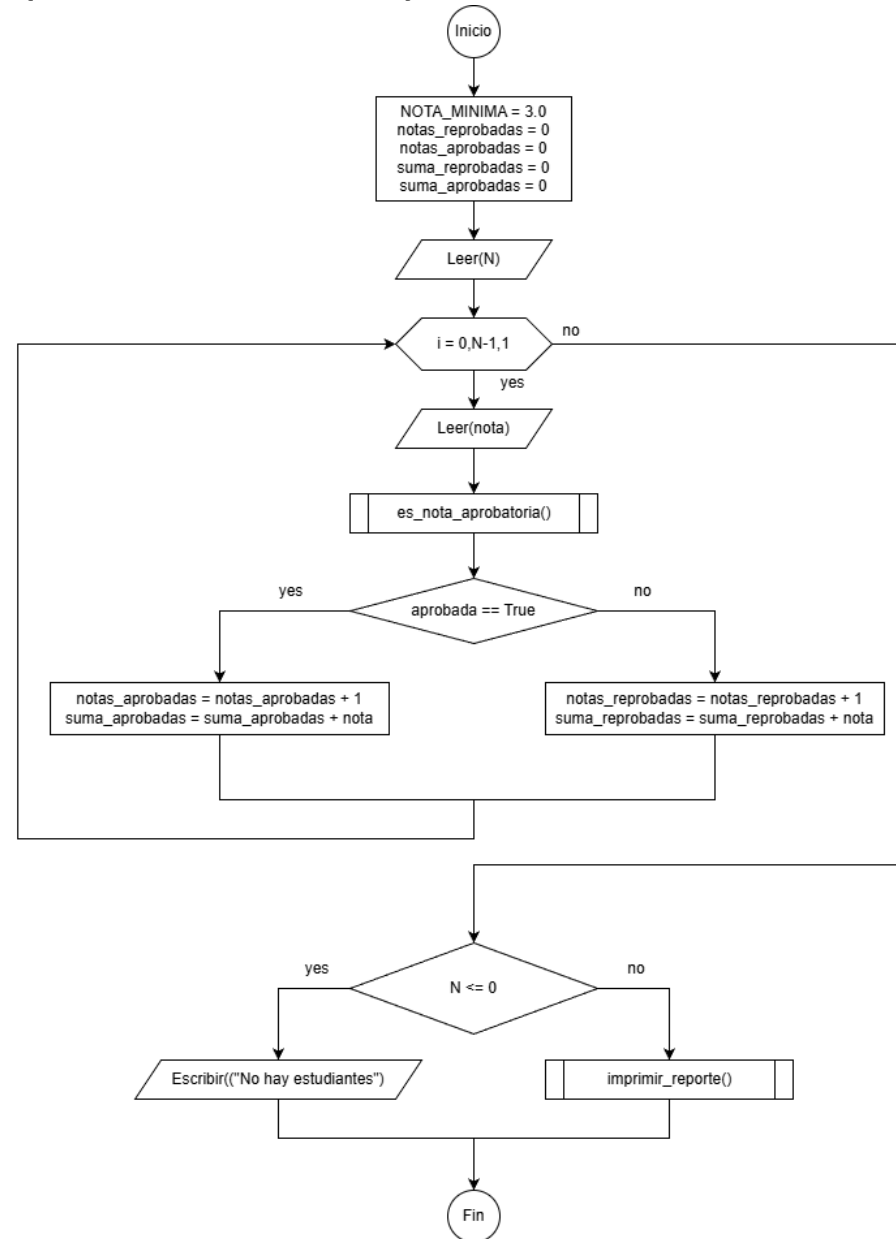
Ejemplo Calificaciones (Modular)

Programa principal

```
# Programa principal
est_aprob = 0
est_reprob = 0
suma_aprob = 0
suma_reprob = 0

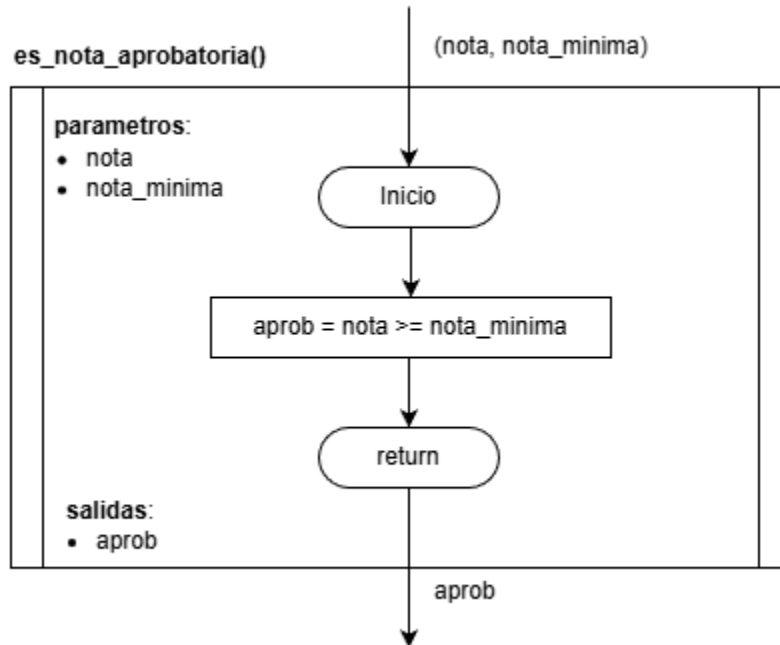
N = int(input("Número de estudiantes: "))
for i in range(N):
    nota = float(input("- Digite la nota: "))
    if es_nota_aprobatoria(nota, NOTA_MINIMA):
        est_aprob += 1
        suma_aprob += nota
    else:
        est_reprob += 1
        suma_reprob += nota

if N <= 0:
    print("No hay estudiantes")
else:
    imprimir_reporte(est_aprob,
                    est_reprob,
                    suma_aprob,
                    suma_reprob)
```



Ejemplo Calificaciones (Modular)

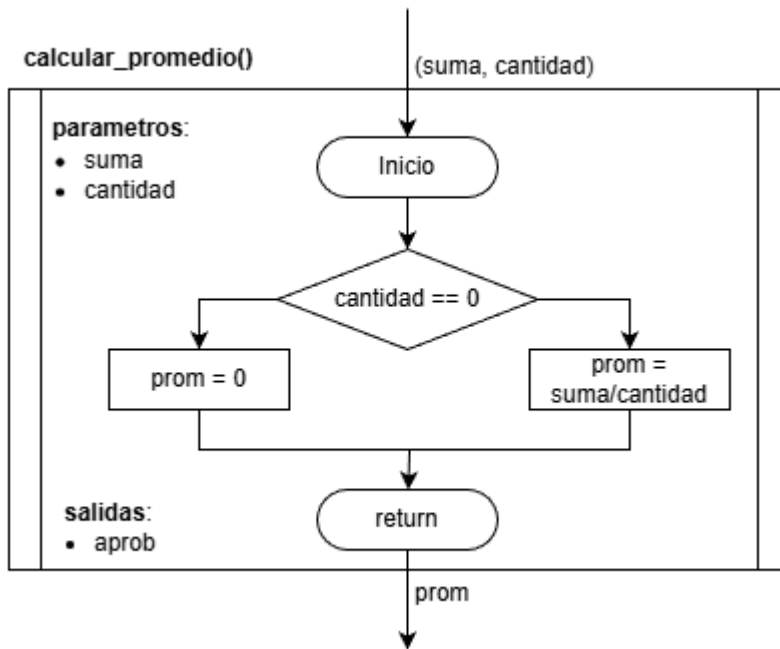
Modulo `es_nota_aprobatoria()`



```
def es_nota_aprobatoria(nota, nota_minima):  
    aprob = nota >= nota_minima  
    return aprob
```

Ejemplo Calificaciones (Modular)

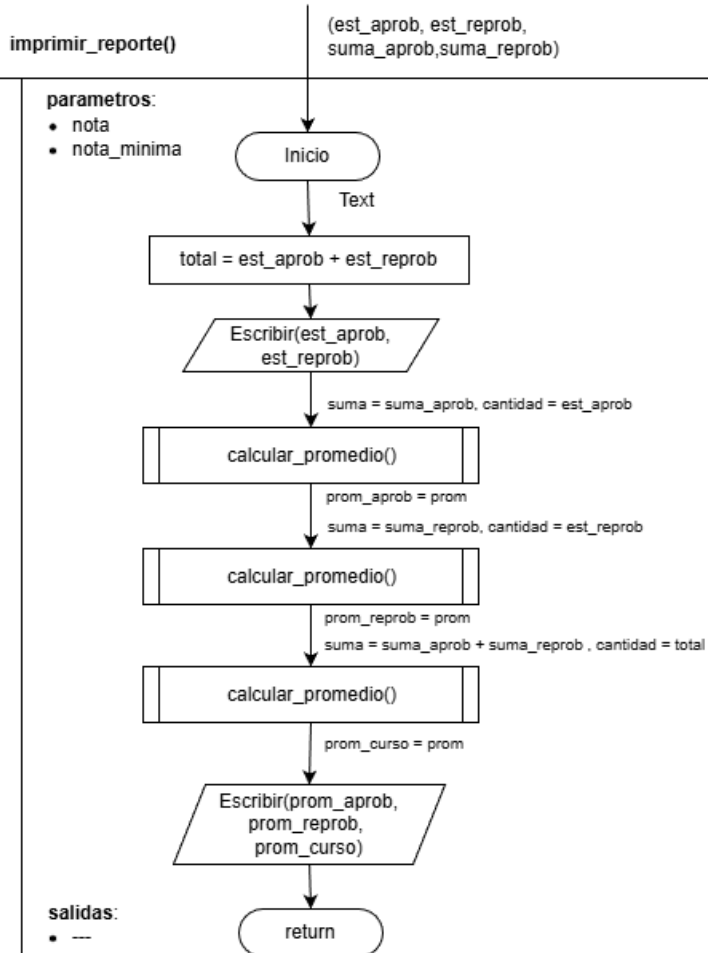
Modulo `calcular_promedio()`



```
def calcular_promedio(suma, cantidad):  
    if cantidad == 0:  
        prom = 0  
    else:  
        prom = suma/cantidad  
    return prom
```

Ejemplo Calificaciones (Modular)

Modulo `imprimir_reporte()`



```
def imprimir_reporte(est_aprob, est_reprob,
                    suma_aprob, suma_reprob):
    total = est_aprob + est_reprob
    print(f"Ganaron: {est_aprob}")
    print(f"Perdieron: {est_reprob}")
    prom_abrob = calcular_promedio(suma_aprob,
                                   est_aprob)
    prom_reprob = calcular_promedio(suma_reprob,
                                    est_reprob)
    prom_curso = calcular_promedio(suma_aprob + suma_reprob,
                                   total)
    print(f"Prom aprob: {prom_aprob:.2f}")
    print(f"Prom reprob: {prom_reprob:.2f}")
    print(f"Prom nota: {prom_curso:.2f}")
```

Ejemplo Calificaciones (Modular)

Codigo completo

```
# Constantes
NOTA_MINIMA = 3.0

# ----- Funciones -----
def es_nota_aprobatoria(nota, nota_minima):
    aprob = nota >= nota_minima
    return aprob

def calcular_promedio(suma, cantidad):
    if cantidad == 0:
        prom = 0
    else:
        prom = suma/cantidad
    return prom

def imprimir_reporte(est_aprob, est_reprob,
                    suma_aprob, suma_reprob):
    total = est_aprob + est_reprob
    print(f"Ganaron: {est_aprob}")
    print(f"Perdieron: {est_reprob}")
    prom_aprob = calcular_promedio(suma_aprob,
                                   est_aprob)
    prom_reprob = calcular_promedio(suma_reprob,
                                    est_reprob)
    prom_curso = calcular_promedio(suma_aprob +
                                   suma_reprob,
                                   total)

    print(f"Prom aprob: {prom_aprob:.2f}")
    print(f"Prom reprob: {prom_reprob:.2f}")
    print(f"Prom nota: {prom_curso:.2f}")
```

```
# ----- Programa principal -----
est_aprob = 0
est_reprob = 0
suma_aprob = 0
suma_reprob = 0

N = int(input("Número de estudiantes: "))
for i in range(N):
    nota = float(input("- Digite la nota: "))
    if es_nota_aprobatoria(nota, NOTA_MINIMA):
        est_aprob += 1
        suma_aprob += nota
    else:
        est_reprob += 1
        suma_reprob += nota

if N <= 0:
    print("No hay estudiantes")
else:
    imprimir_reporte(est_aprob,
                    est_reprob,
                    suma_aprob,
                    suma_reprob)
```

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>
7. <https://realpython.com/defining-your-own-python-function/>
8. <https://realpython.com/python-built-in-functions/>
9. <https://realpython.com/ref/best-practices/functions/>
10. <https://makecode.microbit.org/courses/csintro>
11. <https://makecode.microbit.org/courses>
12. <https://realpython.com/python-while-loop/>
13. <https://realpython.com/ref/keywords/while/>
14. <https://cs.stanford.edu/people/nick/py/>

Eso es todo amigos

